

SIMATS ENGINEERING

ASSESSMENT TOOL 3

NAME :M.GOKUL
REG : 192565067

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

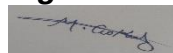
QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I M . G O K U L 1 9 2 5 6 5 0 6 7 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Sequential File vs Heap File Organization

The question asks to compare them based on **insertion, deletion, and retrieval efficiency**.

Feature	Sequential File	Heap File
Insertion	Slow if order must be maintained	Fast
Deletion	Requires locating and maintaining order	Easier
Retrieval	Fast for sequential access	Slower for searching
Organization	Records stored in order	Records stored wherever space is available
Best use	Sequential processing	Frequent insertions

Conclusion: Heap files are better for frequent insertion, while sequential files are better for ordered/sequential retrieval.

2. Clustered vs Non-Clustered Indexing

The question asks for differences with suitable database examples.

Feature	Clustered Index	Non-Clustered Index
Data order	Data stored according to index order	Data order is separate
Number	Usually one per table	Multiple possible
Access	Fast for ordered/range data	Good for specific searches
Example	Employee table ordered by EmployeeID	Index on EmployeeName

Example:

Clustered: EmployeeID → Employee Records

Non-Clustered: EmployeeName → Employee Records

Conclusion: Clustered indexing is useful for range/ordered access, while non-clustered indexing supports additional search fields.

3. Primary Index vs Secondary Indexing

The question asks for comparison based on **storage requirements and search performance**.

Feature	Primary Index	Secondary Index
Based on	Primary key	Non-primary attribute
Storage	Generally less	Generally more
Search	Very efficient	Efficient
Number	Usually one	Multiple possible
Example	StudentID	StudentName

Conclusion: Primary indexing requires less additional storage, while secondary indexing provides more search options but requires extra storage.

4. Static Hashing vs Dynamic Hashing

The question asks to analyze their differences in database systems.

Feature	Static Hashing	Dynamic Hashing
Buckets	Fixed	Can grow/shrink
Database growth	Less suitable	Suitable
Overflow	More likely	Better controlled
Flexibility	Low	High
Performance	Can decrease with growth	More consistent

Conclusion: Static hashing is suitable for fixed-size databases, while dynamic hashing is better when the database size changes.

5. B-Tree vs B+ Tree

The question asks for comparison for **large-scale database applications**.

Feature	B-Tree	B+ Tree
Data storage	Internal and leaf nodes	Leaf nodes
Leaf linking	No	Yes
Range queries	Less efficient	Very efficient
Search	Fast	Fast
Database use	General indexing	Large databases

Conclusion: B+ Tree is generally preferred for large-scale databases because linked leaf nodes make sequential and range retrieval efficient.

6. Linear Hashing vs Extendible Hashing

The question asks specifically about **bucket management and scalability**.

Feature	Linear Hashing	Extendible Hashing
Bucket growth	Gradual	Bucket splitting with directory
Directory	No large directory required	Uses directory
Scalability	Good	Very good
Overflow handling	Controlled splitting	Directory/bucket splitting
Complexity	Simpler	More complex

Conclusion: Linear hashing grows gradually, while extendible hashing provides flexible growth using directory expansion.

7. Dense vs Sparse Indexing

The question asks for comparison based on **memory usage and access speed**.

Feature	Dense Index	Sparse Index
Entries	Every record	Selected records
Memory	High	Low
Access speed	Faster	Relatively slower
File requirement	Can work with indexed records	Usually requires sorted file
Storage	More	Less

Conclusion: Dense indexing provides faster access but consumes more memory. Sparse indexing saves memory but may require additional searching.

8. Indexed Sequential vs Direct File Organization

The question asks for **advantages and limitations** of both organizations.

Feature	Indexed Sequential	Direct
Access	Sequential + indexed	Direct
Search	Efficient	Very fast for exact key
Range queries	Good	Less suitable
Complexity	Higher	Simpler
Storage	Requires index	Hash/address structure

Indexed Sequential

Advantages:

- Supports sequential access.
- Supports direct indexed access.
- Good for range queries.

Limitations:

- Requires additional index storage.
- Maintaining index has overhead.

Direct

Advantages:

- Fast direct access.
- Good for exact-match queries.

Limitations:

- Poor for sequential/range processing.
- Collision handling may be required.

Conclusion: Indexed sequential is better for both sequential and range access, while direct organization is better for fast exact access.

9. Single-Level vs Multi-Level Indexing

The question asks to compare them for efficient database retrieval.

Feature	Single-Level	Multi-Level
Index levels	One	Multiple
Large database	Less efficient	More efficient
Search	Direct index search	Searches through index levels
Storage	Simpler	More structured
Access	More disk accesses for large indexes	Reduces disk accesses

Example:

Single-Level:

Index → Data

Multi-Level:

Level 1 → Level 2 → Data

Conclusion: Multi-level indexing is more suitable for large databases because it reduces the number of disk accesses.

10. Hashing vs Indexing

The question asks to evaluate performance for **exact-match and range queries**.

Query Type	Hashing	Indexing
Exact-match	Very fast	Fast
Range query	Poor	Very good
Equality search	Excellent	Excellent
Ordered retrieval	Not suitable	Suitable
Example	StudentID = 101 CGPA BETWEEN 8 AND 9	

Hashing

Best for:

StudentID = 101

AccountNo = 5001

Average exact search is approximately **O(1)**.

Indexing

Best for:

CGPA BETWEEN 8.0 AND 9.0

Date BETWEEN Jan 1 AND Jan 31

B+ Tree indexing is particularly suitable for range queries.

Conclusion: Hashing is better for exact-match queries, while indexing is better for

range queries.
